



UNIVERSITY OF
ILLINOIS
URBANA-CHAMPAIGN

SE 423: Introduction to Mechatronics

Lecture 18: Path planning

Marius Juston

DATE

Talk to the person next to you and answer these questions.

- If you had multi-goals that you were trying to reach what would use A^* , Dijkstra, Greedy Best First, or BFS? What about a single goal?
- In the previous lecture we performed path planning on a grid graph, what did each vertex in the grid represent (i.e. what was the state space that A^* was searching through?)
- What kind robot is most closely represented when you perform path planning with the state only being (x, y) ?
- For a differential robot, is the movement cost uniform based purely on the position on the robot? The movement cost from $(x, y) \rightarrow (x + 1, y) =$ or $\neq (x, y) \rightarrow (x, y + 1)$
- What is the state space of differential robot?

Office Hours:

- **Monday:** 4-6 PM, Neel
- **Tuesday:** 11-1 PM, Lakshmi
- **Tuesday:** 1-3 PM, Samuel
- **Tuesday:** 2-5 PM, Dan & Marius

Lab: Starting Lab 6 this week. This a 3-week lab!

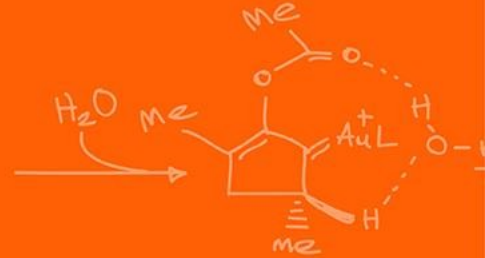
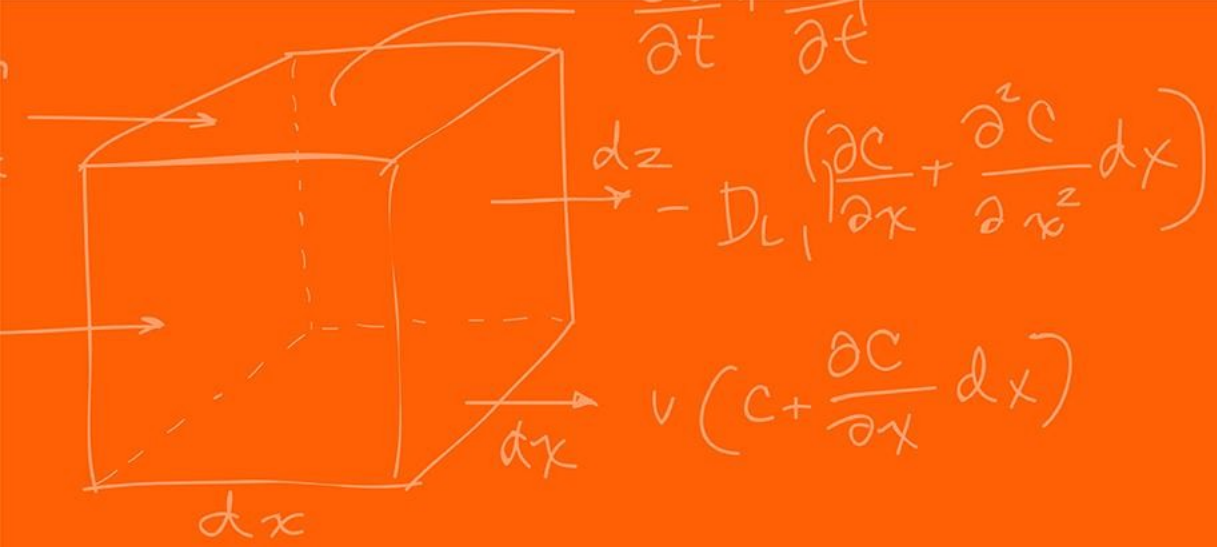
- Don't forget to have the Lab LabVIEW application implemented before lab.

Homeworks:

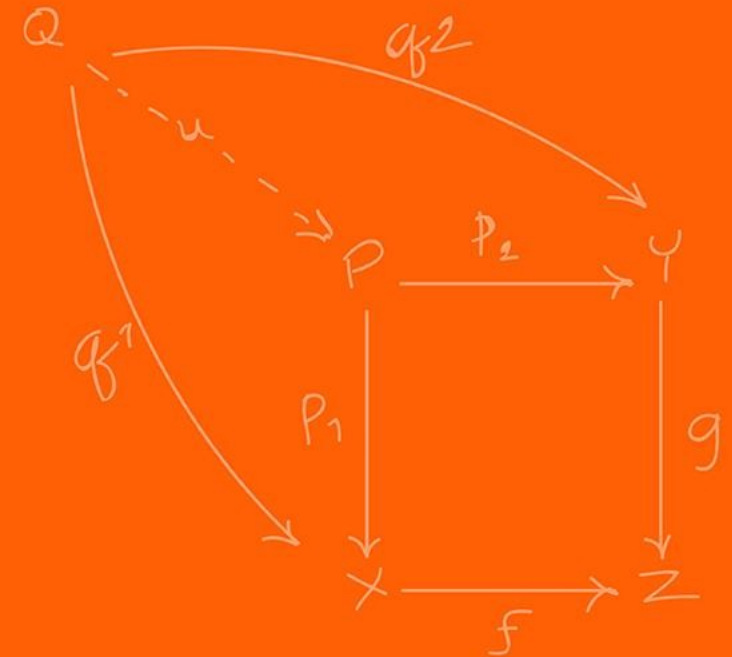
- Check-offs for [homework 4](#) Ex 4 are due **April 7, 5PM (The end of office hours)**
- Remainder of [homework 4](#) due on Gradescope on **April 8, 9AM**
- Next homework is primarily the **personal project** due on **April 22, 9AM**.
 - I would highly recommend you thinking now and maybe even starting it now if you have a good idea already!

Questions?

Homework, Lab, LabVIEW?



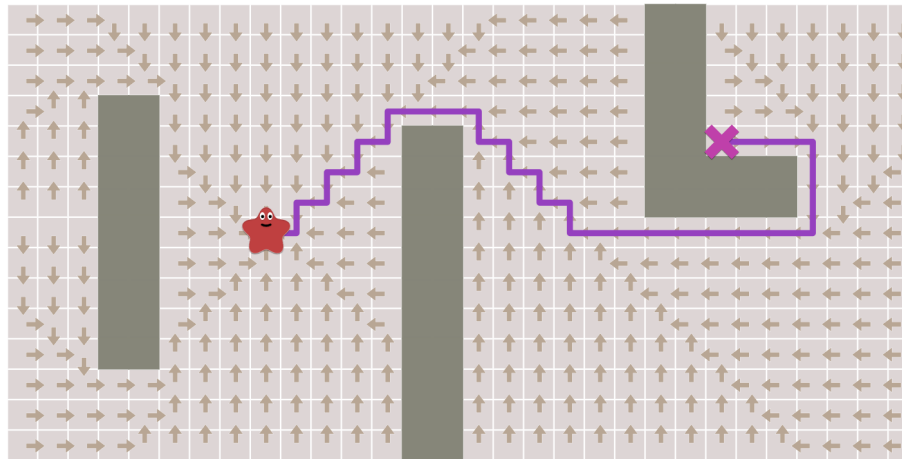
Path planning



In the previous lecture we explored A* on a 2D grid, where we represented the environment as a graph discrete nodes.

What was the output path returned?

The output path was a sequence of (x, y) waypoints.



In lecture 15, we however, define the robot's pose $p = (x, y, \theta)$ in the global frame which moves using the linear velocity V and angular ω .

Standard A* assumes the robot is a "point mass" that can move in any direction!

However, differential drive robots cannot translate sideways! A drone, however, would be fine with this "point mass" definition.

When the path goes from moving "North" to "East", the angle changes by 90 degrees instantly.

This implies an infinite angular velocity at that corner if you are not already facing that direction. What happens to our DC motors if we command an infinite angular velocity?

How can we handle this?

Stop and turn or try tracking the next waypoint.

One common approach is to ignore the robot's kinematics during planning.

1. Plan in 2D (x, y) space using A^* .
2. Build a continuous feedback controller to "chase" the discrete path.
3. We infer the desired heading θ_r from the angle between waypoints:

$$\theta_r = \text{atan2}(\Delta y, \Delta x)$$

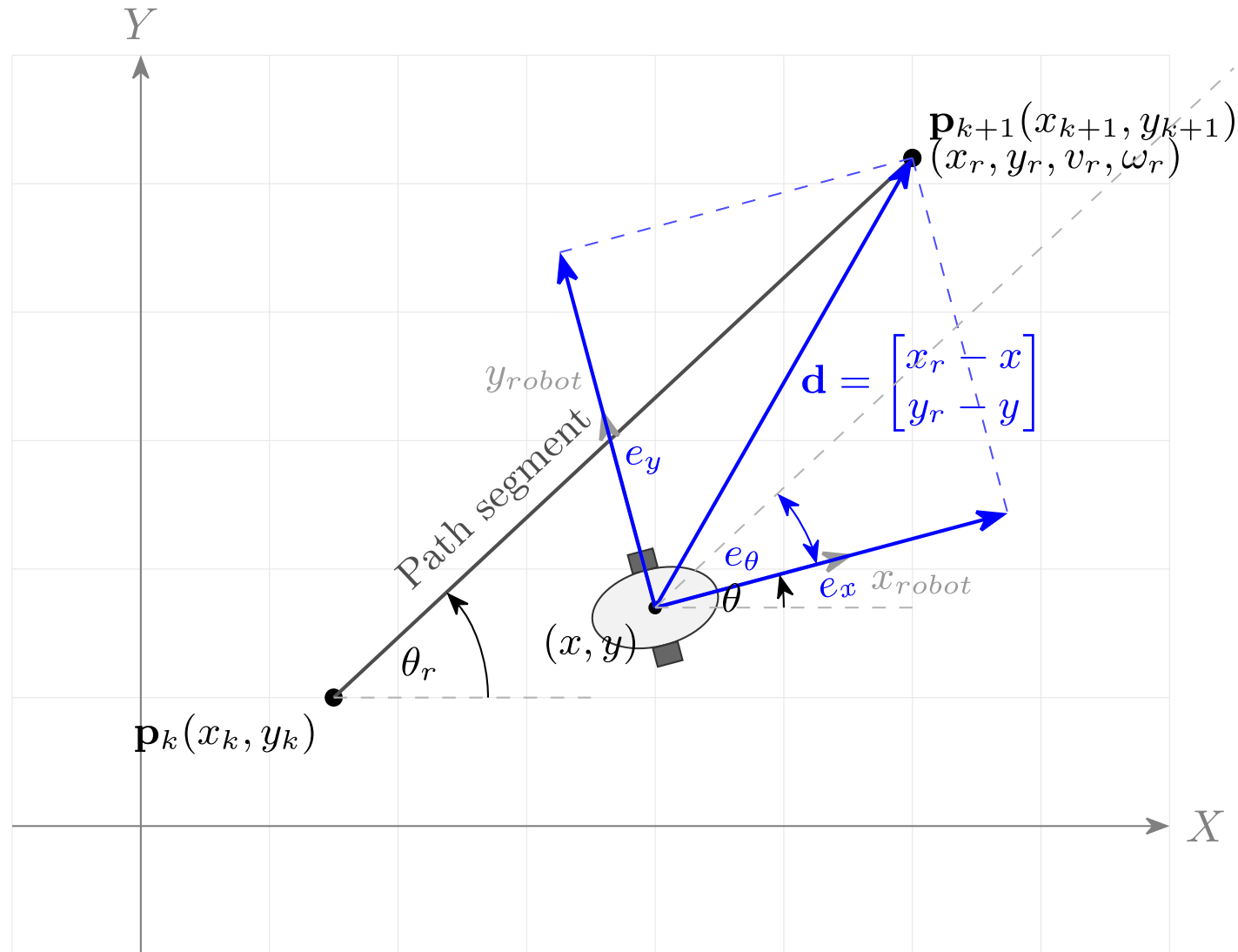
One possible way is to define the error state $\mathbf{e} = [e_x, e_y, e_\theta]^T$ transformed into the robot's local coordinate frame:

$$\begin{bmatrix} e_x \\ e_y \\ e_\theta \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r - x \\ y_r - y \\ \theta_r - \theta \end{bmatrix}$$

Where (x_r, y_r, θ_r) , represents the waypoint coordinate from A^* .

The kinematic model governing the motion of a unicycle (which accurately represents the center point of a differential drive robot) is:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & 0 \\ \sin(\theta) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$



Taking the time derivative of this error vector and substituting our kinematic equations yields the open-loop error dynamics:

$$\begin{aligned} \dot{e}_x &= \omega e_y - v + v_r \cos(e_\theta) \\ \dot{e}_y &= -\omega e_x + v_r \sin(e_\theta) \\ \dot{e}_\theta &= \omega_r - \omega \end{aligned}$$

To generate control actions that guarantee the robot will converge to the path, we can utilize a non-linear feedback tracking controller (originally proposed by Kanayama et al.).

We propose the following control law using positive gain constants $K_x, K_y, K_\theta > 0$:

$$\begin{aligned} v &= v_r \cos(e_\theta) + K_x e_x \\ \omega &= \omega_r + v_r (K_y e_y + K_\theta \sin(e_\theta)) \end{aligned}$$

With non-linear control stability theory (Lyapunov), you can prove that the previously provided controller will converge to the path.

You can define a positive-definite Lyapunov candidate function $V(\mathbf{e})$:

$$V = \frac{1}{2}(e_x^2 + e_y^2) + \frac{1 - \cos(e_\theta)}{K_y}$$

We must prove that its time derivative $\dot{V}$ is negative semi-definite. Taking the derivative:

$$\dot{V} = e_x \dot{e}_x + e_y \dot{e}_y + \frac{\sin(e_\theta)}{K_y} \dot{e}_\theta$$

I will leave as an exercise to you to verify that as long as $K_x, K_y, K_\theta > 0$ and $v_r > 0$. The error states asymptotically converge to zero.

So, we have decoupled the control from the robot's planner. However, doing so we could have potential problems. What might they be?

When inferring θ from a jagged grid path, the reference heading θ_r is discontinuous.

This creates sudden spikes in heading error (e_θ).

The feedback controller saturates the motors trying to fix the error.

The robot swings wide around corners, potentially crashing into obstacles the planner thought were safe.

The path planner does not understand the robot's kinematics when planning!

⌚ To fix this, the planner must understand the robot's physical heading.

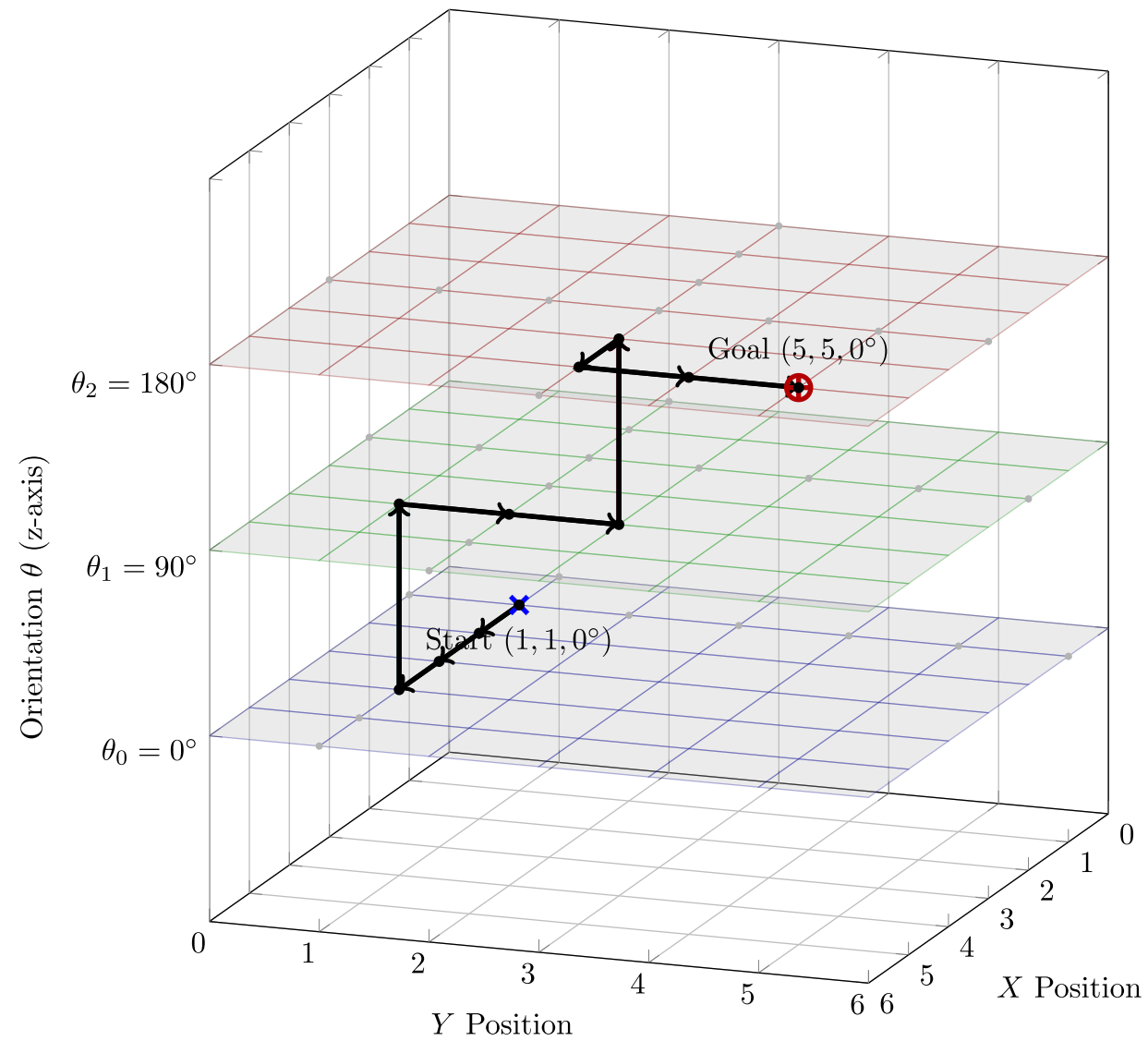
We must expand our search state from 2D (x, y) to 3D (x, y, θ) .

The robot now searches through positions and orientations! A state transition is no longer just moving to an adjacent grid cell.

We can discretize θ into bins (e.g., 8 directions, 16 directions).

From a state $(x, y, 0^\circ)$, valid moves might be "drive forward at 0° " or "turn to 45° and drive".

This ensures the path considers the robot's current orientation. However, true non-holonomic robots don't just "turn and drive"—they curve.



Adding θ dramatically increases the search space.

If a 100x100 grid has 10,000 states in 2D discretizing θ into 72 angles (every 5 degrees) yields 720,000 states!

How does this affect the memory and compute time required for A*?

Every time you add a dimension you increase the number of possibilities exponentially!

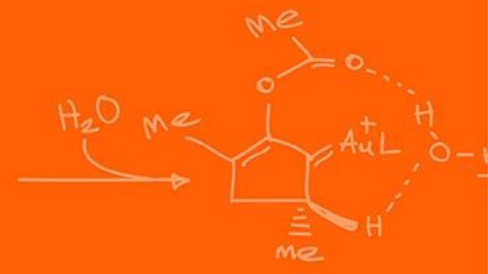
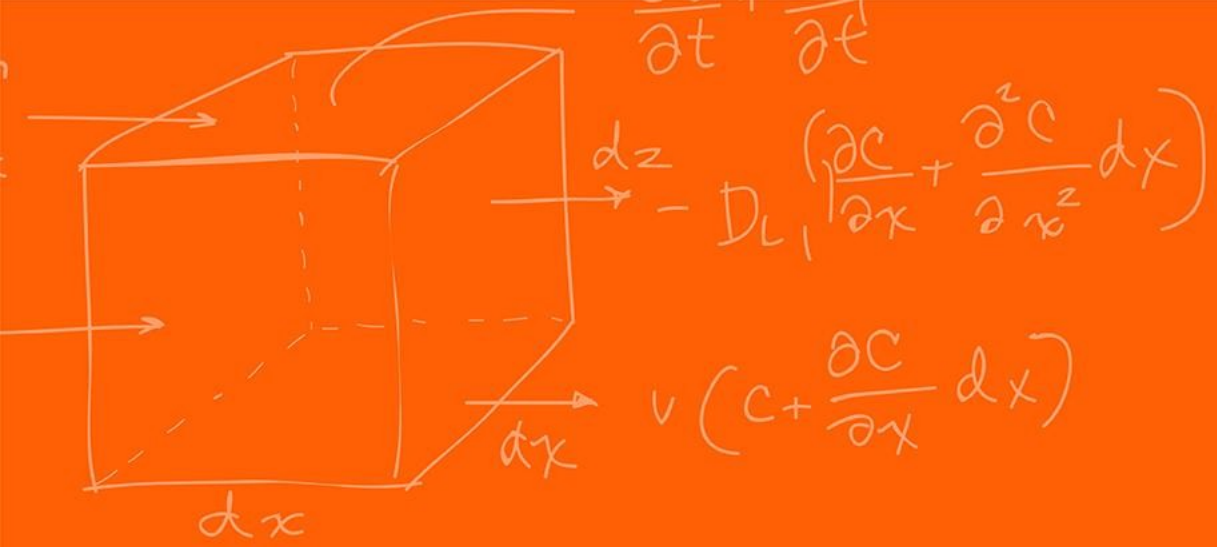
This phenomenon is called the **Curse of Dimensionality** and is a problem in a lot of applications of math!

There is no way to escape this problem!

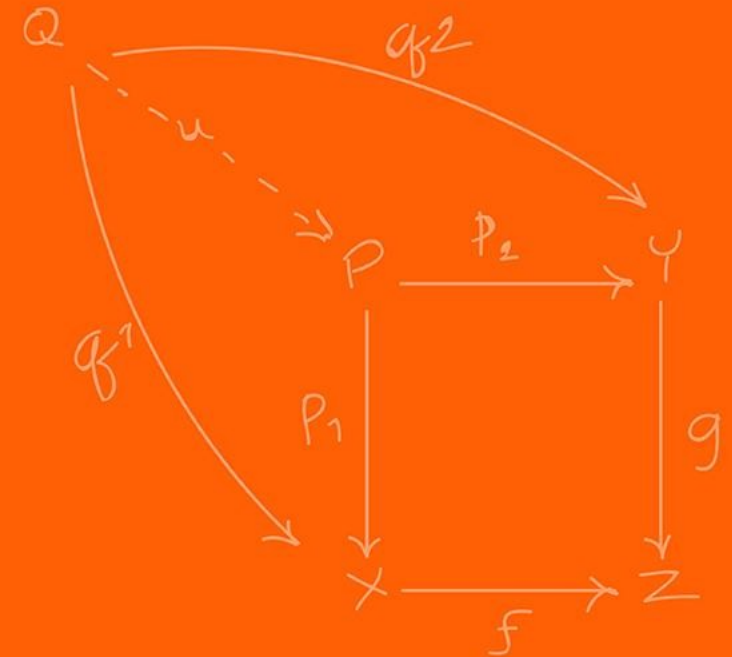
Before we commit to heavy 3D planning, let's establish when 2D is acceptable:

- 2D planning works well for omnidirectional robots (Mecanum wheels).
- It also works for very slow differential drive robots in wide-open spaces (you can have 8-connected neighbors and tune your controller to minimize edge clipping).

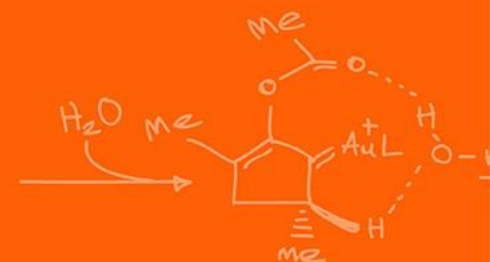
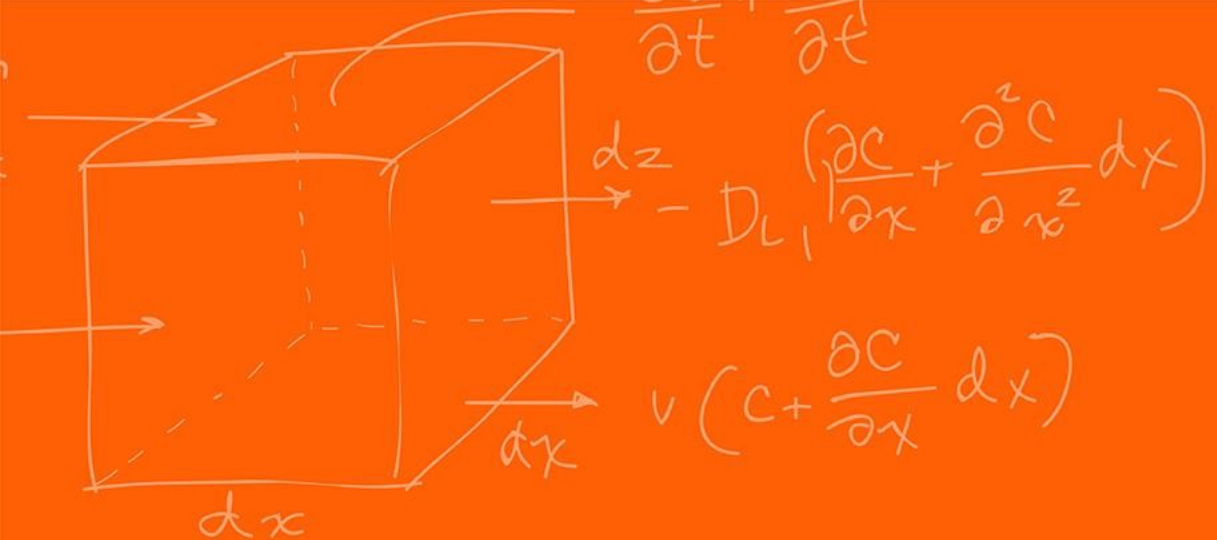
It fails catastrophically for cars (Ackermann steering) or robots in tight, cluttered corridors.



Hybrid A*



Questions?



The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

